In [1]:

```
import networkx as nx
import matplotlib.pyplot as plt

G = nx.DiGraph()

G.add_nodes_from(['A', 'B', 'C', 'D', 'E'])

G.add_edges_from([
    ('A', 'B'),
    ('B', 'C'),
    ('C', 'D'),
    ('C', 'E'),
    ('E', 'D'),
    ('C', 'B')
])

print("Adjacency List:")
for node in G.nodes():
    print(node, ":", list(G.neighbors(node)))

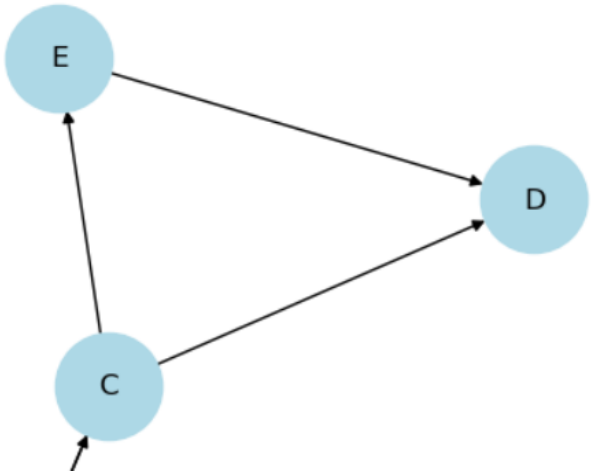
print("\nAdjacency Matrix:")
print(nx.to_numpy_array(G, nodelist=['A', 'B', 'C', 'D', 'E'], dtype=int))

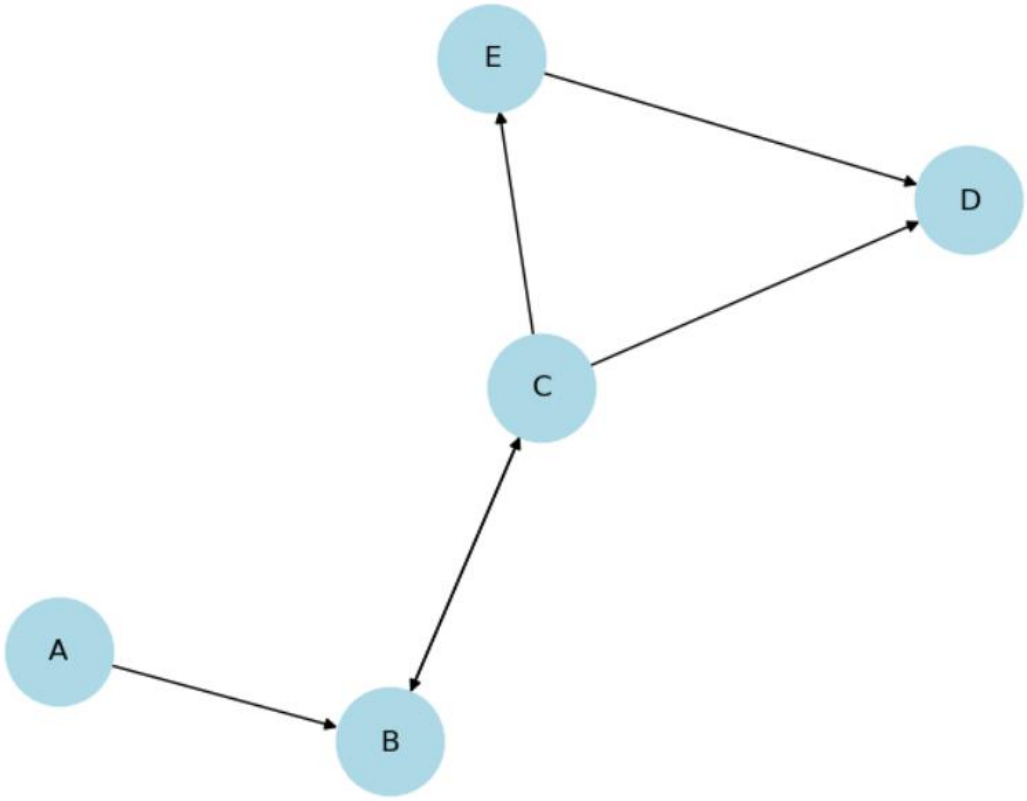
nx.draw(G, with_labels=True, node_color='lightblue',
        node_size=2000, arrows=True)
plt.show()
```

```
Adjacency List:
A : ['B']
```

```
Adjacency List:
A : ['B']
B : ['C']
C : ['D', 'E', 'B']
D : []
E : ['D']

Adjacency Matrix:
[[0 1 0 0 0]
 [0 0 1 0 0]
 [0 1 0 1 1]
 [0 0 0 0 0]
 [0 0 0 1 0]]
```




E
D
C
A
B

In [2]:

```python
import networkx as nx

G = nx.DiGraph()

G.add_edges_from([
    ('A', 'B'),
    ('B', 'C'),
    ('C', 'D'),
    ('C', 'E'),
    ('E', 'D'),
    ('C', 'B')
])

print("In-degree and Out-degree:")

for node in G.nodes():
    print(node, "In-degree:", G.in_degree(node),
          "Out-degree:", G.out_degree(node))

print("Self-loops:", list(nx.selfloop_edges(G)))
```

```
In-degree and Out-degree:
A In-degree: 0 Out-degree: 1
B In-degree: 2 Out-degree: 1
C In-degree: 1 Out-degree: 3
D In-degree: 2 Out-degree: 0
E In-degree: 1 Out-degree: 1
Self-loops: []
```

In [3]:
```python
import networkx as nx

G = nx.DiGraph()

G.add_edges_from([
    ('A', 'B'),
    ('B', 'C'),
    ('C', 'D'),
    ('C', 'E'),
    ('E', 'D'),
    ('C', 'B')
])

if nx.has_path(G, 'A', 'D'):
    print("D is reachable from A")

    print("Simple paths from A to D:")
    for path in nx.all_simple_paths(G, 'A', 'D'):
        print(path)
else:
    print("D is not reachable from A")

print("Cycles:", list(nx.simple_cycles(G)))
```

```
D is reachable from A
Simple paths from A to D:
['A', 'B', 'C', 'D']
['A', 'B', 'C', 'E', 'D']
Cycles: [['C', 'B']]
```

In [4]:
```
import networkx as nx
import matplotlib.pyplot as plt

G = nx.DiGraph()

G.add_nodes_from([1, 2, 3, 4, 5, 6, 7, 8])

G.add_edges_from([
    (1, 2), (1, 3), (2, 4), (4, 2),
    (2, 5), (3, 5), (5, 3), (4, 6),
    (5, 6), (6, 7), (7, 8), (8, 6)
])

pos = nx.spring_layout(G, seed=42)

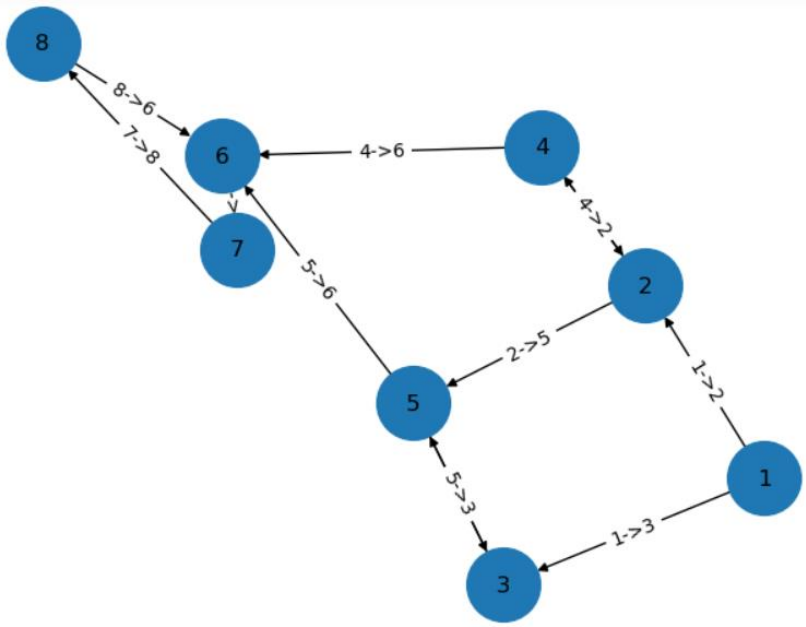
nx.draw(
    G, pos, with_labels=True,
    node_size=1500, arrows=True
)

nx.draw_networkx_edge_labels(
    G, pos,
    edge_labels={(u, v): f"{u}->{v}" for u, v in G.edges()}
)

plt.show()

print("Adjacency List:")
for node in G.nodes():
    print(node, ":", list(G.neighbors(node)))

print("\nAdjacency Matrix:")
```




```
Adjacency List:
1 : [2, 3]
2 : [4, 5]
```

```
Adjacency List:
1 : [2, 3]
2 : [4, 5]
3 : [5]
4 : [2, 6]
5 : [3, 6]
6 : [7]
7 : [8]
8 : [6]

Adjacency Matrix:
[[0 1 1 0 0 0 0 0]
 [0 0 0 1 1 0 0 0]
 [0 0 0 0 1 0 0 0]
 [0 1 0 0 0 1 0 0]
 [0 0 1 0 0 1 0 0]
 [0 0 0 0 0 0 1 0]
 [0 0 0 0 0 0 0 1]
 [0 0 0 0 0 1 0 0]]
```

In [5]:

```python
import networkx as nx

G = nx.DiGraph()

G.add_edges_from([
    (1, 2), (1, 3), (2, 4), (4, 2),
    (2, 5), (3, 5), (5, 3), (4, 6),
    (5, 6), (6, 7), (7, 8), (8, 6)
])

print("Cycles:")
print(list(nx.simple_cycles(G)))
```

```
Cycles:
[[2, 4], [3, 5], [8, 6, 7]]
```

In [6]:

```python
G.remove_edge(4, 2)
G.remove_edge(5, 3)
G.remove_edge(8, 6)

print("Remaining cycles:", list(nx.simple_cycles(G)))
```

```
Remaining cycles: []
```

In [7]:
```
for node in range(1, 8):
    path = nx.shortest_path(G, source=node, target=8)
    print("Shortest path from", node, "to 8:", path)
```

```
Shortest path from 1 to 8: [1, 2, 4, 6, 7, 8]
Shortest path from 2 to 8: [2, 4, 6, 7, 8]
Shortest path from 3 to 8: [3, 5, 6, 7, 8]
Shortest path from 4 to 8: [4, 6, 7, 8]
Shortest path from 5 to 8: [5, 6, 7, 8]
Shortest path from 6 to 8: [6, 7, 8]
Shortest path from 7 to 8: [7, 8]
```

In [8]:
```
import networkx as nx
import matplotlib.pyplot as plt

G = nx.DiGraph()

G.add_nodes_from([
    'Sol', 'Alpha', 'Vega', 'Sirius', 'Betelgeuse'
])

G.add_edges_from([
    ('Sol', 'Alpha'),
    ('Alpha', 'Vega'),
    ('Vega', 'Sirius'),
    ('Vega', 'Betelgeuse'),
    ('Betelgeuse', 'Sirius'),
    ('Vega', 'Alpha')
])

print("Adjacency List:")
for node in G.nodes():
    print(node, ":", list(G.neighbors(node)))

nodes = ['Sol', 'Alpha', 'Vega', 'Sirius', 'Betelgeuse']

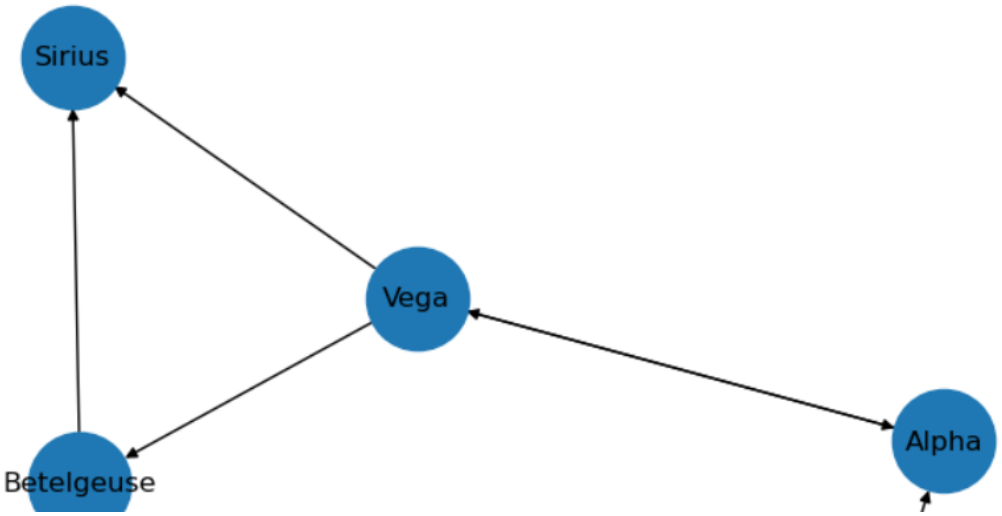
print("\nAdjacency Matrix:")
print(nx.to_numpy_array(G, nodelist=nodes, dtype=int))

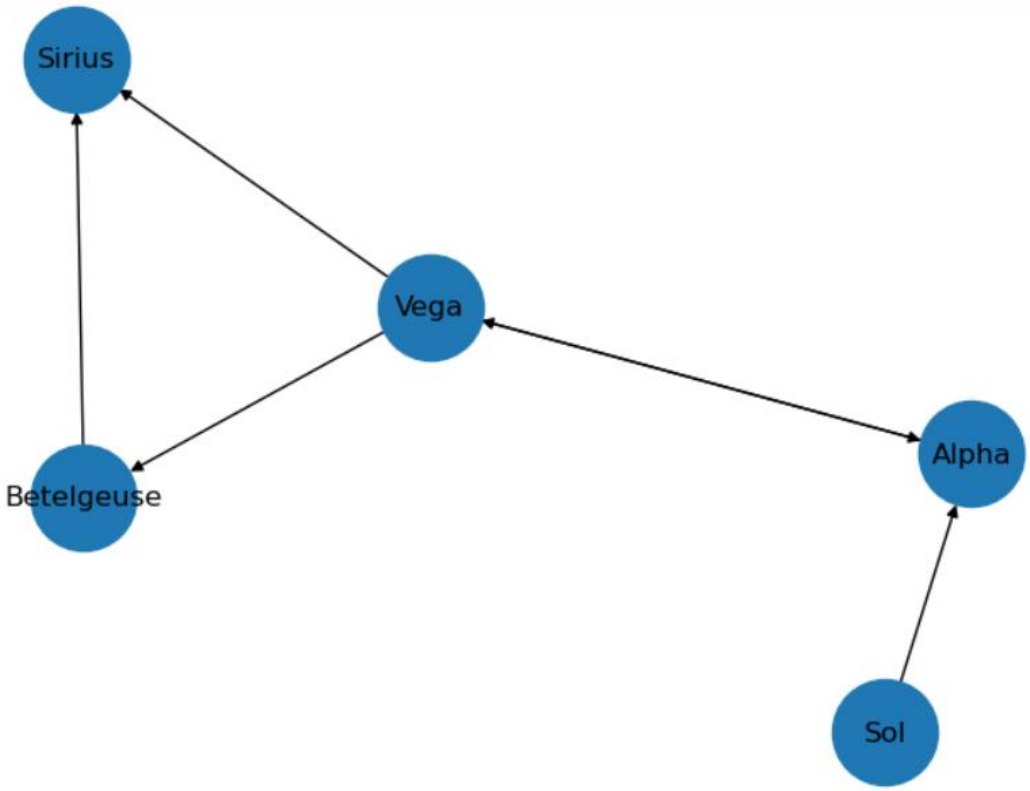
nx.draw(G, with_labels=True, node_size=2000, arrows=True)
plt.show()
```

```
Adjacency List:
```

```
Sol : ['Alpha']
Alpha : ['Vega']
Vega : ['Sirius', 'Betelgeuse', 'Alpha']
Sirius : []
Betelgeuse : ['Sirius']

Adjacency Matrix:
[[0 1 0 0 0]
 [0 0 1 0 0]
 [0 1 0 1 1]
 [0 0 0 0 0]
 [0 0 0 1 0]]
```




Sirius
Vega
Alpha
Betelgeuse
Sol

In [9]:

```
import networkx as nx

G = nx.DiGraph()

G.add_edges_from([
    ('Sol', 'Alpha'),
    ('Alpha', 'Vega'),
    ('Vega', 'Sirius'),
    ('Vega', 'Betelgeuse'),
    ('Betelgeuse', 'Sirius'),
    ('Vega', 'Alpha')
])

print("In-degree and Out-degree:")

for node in G.nodes():
    print(node,
          "In-degree:", G.in_degree(node),
          "Out-degree:", G.out_degree(node))

print("Self-loops:", list(nx.selfloop_edges(G)))
```

```
In-degree and Out-degree:
Sol In-degree: 0 Out-degree: 1
Alpha In-degree: 2 Out-degree: 1
Vega In-degree: 1 Out-degree: 3
Sirius In-degree: 2 Out-degree: 0
Betelgeuse In-degree: 1 Out-degree: 1
Self-loops: []
```

In [10]:
```python
import networkx as nx

G = nx.DiGraph()

G.add_edges_from([
    ('Sol', 'Alpha'),
    ('Alpha', 'Vega'),
    ('Vega', 'Sirius'),
    ('Vega', 'Betelgeuse'),
    ('Betelgeuse', 'Sirius'),
    ('Vega', 'Alpha')
])

if nx.has_path(G, 'Sol', 'Sirius'):
    print("Simple paths from Sol to Sirius:")

    for path in nx.all_simple_paths(G, 'Sol', 'Sirius'):
        print(path)

print("Cycles:", list(nx.simple_cycles(G)))
```

```
Simple paths from Sol to Sirius:
['Sol', 'Alpha', 'Vega', 'Sirius']
['Sol', 'Alpha', 'Vega', 'Betelgeuse', 'Sirius']
Cycles: [['Vega', 'Alpha']]
```

In [11]:
```python
import networkx as nx
import matplotlib.pyplot as plt

G = nx.DiGraph()

G.add_nodes_from(['S1', 'S2', 'S3', 'S4', 'S5'])

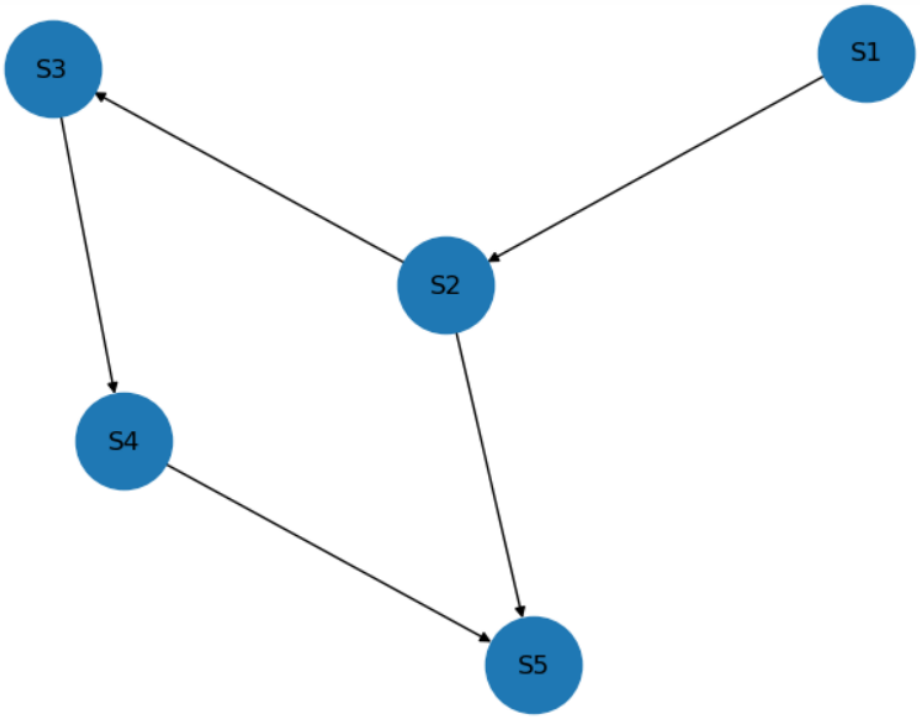
G.add_edges_from([
    ('S1', 'S2'),
    ('S2', 'S3'),
    ('S3', 'S4'),
    ('S4', 'S5'),
    ('S2', 'S5')
])

nx.draw(G, with_labels=True, node_size=2000, arrows=True)
plt.show()

nodes = ['S1', 'S2', 'S3', 'S4', 'S5']

matrix = nx.to_numpy_array(G, nodelist=nodes, dtype=int)

print("Adjacency Matrix:")
print(matrix)
```




```
Adjacency Matrix:
[[0 1 0 0 0]
 [0 0 1 0 1]
 [0 0 0 1 0]
 [0 0 0 0 1]
 [0 0 0 0 0]]
```

In [12]:

```
import networkx as nx

G = nx.DiGraph()

G.add_edges_from([
    ('S1', 'S2'),
    ('S2', 'S3'),
    ('S3', 'S4'),
    ('S4', 'S5'),
    ('S2', 'S5')
])

for node in G.nodes():
    print(node,
          "In-degree:", G.in_degree(node),
          "Out-degree:", G.out_degree(node))

if nx.has_path(G, 'S1', 'S5'):
    print("S5 is reachable from S1")
    print("Path:", nx.shortest_path(G, 'S1', 'S5'))
else:
    print("S5 is not reachable from S1")
```

```
S1 In-degree: 0 Out-degree: 1
S2 In-degree: 1 Out-degree: 2
S3 In-degree: 1 Out-degree: 1
S4 In-degree: 1 Out-degree: 1
S5 In-degree: 2 Out-degree: 0
S5 is reachable from S1
Path: ['S1', 'S2', 'S5']
```